QA EKS MENTOR: ARCHITECTURE & METHODOLOGY

Automated Root-Cause Analysis (RCA) Engine for Whitebox Static Tool Validation

Model: Qwen 2.5 3B (QLoRA) | Quantization: Q4_K_M (1.93 GB) | Runtime: Ollama Local GPU | Suite: Playwright Tool Validation

1. EXECUTIVE SUMMARY & PROBLEM SPACE

During Whitebox automated tool validation, static tools (such as `crosshair`, `spotbugs`, or `pylint`) frequently fail to trigger. Historically, uncovering the cause required manual inspection of thousands of Kubernetes logs across 9 microservices in EKS cell-001. **QA EKS Mentor** eliminates this bottleneck by automating log extraction, root-cause deduction, and Jira/PDF reporting.

< 5s	100%	$0.00	0
DIAGNOSIS LATENCY (VS. 40M MANUAL)	LOCAL GPU OFFLOAD (RTX 3050)	CLOUD INFERENCE & API COST	HALLUCINATIONS (STRICT LOG RULES)

❌ Before (Manual Log Hunting)

- 30–45 minutes of manual `kubectl logs` digging per failure.
- Generic ticket descriptions (e.g. "*Check compatibility.py*").
- Frequent back-and-forth arguments over repository vs. platform fault.
- Developer frustration and delayed release cycles.

✅ After (QA EKS Mentor)

- Instant root cause extraction directly from orchestrator planning events.
- Exact log citations (timestamp, logger, and error reason quoted).
- Clear ownership assigned (Platform Orchestrator vs. Customer Repo).
- Executive 2-page PDF & 7-section Jira markdown created automatically.

2. THE 4-PILLAR AI METHODOLOGY

Pillar 1: QLoRA Fine-Tuning (Cloud Free-Tier GPU)
Rather than full retraining (which requires $30k enterprise clusters), we used **QLoRA (Quantized Low-Rank Adaptation)**. We quantized `Qwen 2.5 3B` to 4-bit precision and attached small, trainable LoRA adapter matrices (~1% of parameters). Training was executed on a free Google Colab 16 GB Nvidia T4 GPU via Unsloth in under 10 minutes.

Pillar 2: The "Zero Hallucination" Iron Law
Every claim in the root-cause diagnosis must cite a literal log line from the EKS evidence block. If logs are outside the retention window, the model is strictly conditioned to declare `Verdict: Insufficient evidence` rather than fabricating causes.

Pillar 3: The Hybrid Division of Labor
Deterministic code (TypeScript and Python) extracts raw JSON logs, verifies pod states, and matches known error codes. The fine-tuned LLM synthesizes these verified facts into an executive English summary and concrete remediation steps.

Pillar 4: Local GGUF Deployment (100% GPU)
The trained weights were merged and exported to `Q4_K_M GGUF` (1.93 GB) and loaded into Ollama. The model runs 100% inside your laptop's NVIDIA GeForce RTX 3050 GPU VRAM (2.2 GB footprint), ensuring total privacy and offline reliability.

3. END-TO-END SYSTEM EXECUTION FLOW

How the Playwright validation suite communicates with the local AI model during a test run:

Sequence of Operations:

1. **Playwright Test:** `merge-flow.spec.ts` tests a repository branch in the Whitebox portal, downloading `run-details.json` and taxonomy gate HTML.
2. **S3 Verification:** `s3Evidence.ts` checks S3 tool output buckets. If an expected tool (e.g. `crosshair`) is missing, it logs a `[Planning Defect]`.
3. **AI Consultation:** `bugReport.ts` triggers `eksMentor.ts`, which queries `http://localhost:11434/api/chat` (model: `qa-eks-mentor-trained`).
4. **RCA Deduction:** The model returns the verified root cause, platform owner, and an executive summary. (If offline, it safely falls back to default rules).
5. **Artifact Delivery:** Playwright captures a 2x taxonomy screenshot, compiles the 2-page Executive PDF, and generates the 7-section Jira markdown file.
6. **Publishing:** `postprocess.ts` copies the artifacts to SharePoint and sends an adaptive card to Microsoft Teams.

4. CORE FILE INVENTORY & ARCHITECTURAL ROLES

Module / File	Language / Tech	Architectural Function
`utils/eksMentor.ts`	TypeScript (Node)	The client bridge. Queries local Ollama REST API with safe timeouts and non-blocking fallbacks.
`utils/bugReport.ts`	TypeScript (Playwright)	Renders the 2-page Executive Defect PDF and 7-section Jira markdown with AI citations.
`qa-eks-mentor-trained`	Ollama GGUF (4-bit)	The fine-tuned model (1.93 GB) running on the local NVIDIA RTX 3050 GPU.
`mentor/analyze.py`	Python 3	Direct Kubernetes utility pulling live logs across 9 microservices via `kubectl`.
`mentor/Modelfile`	Ollama Spec	Packages the GGUF weights, ChatML templates, `temperature 0.1`, and QA persona.
`training/prepare_dataset.py`	Python 3	Formats raw EKS log runs into ChatML JSONL pairs for future training iterations.
`training/qa_eks_mentor_training.ipynb`	Jupyter / Colab	Automated Unsloth QLoRA notebook for cloud GPU training and GGUF quantization.

5. BUSINESS VALUE & IMPACT SUMMARY

- **Drastic Defect Cycle Acceleration:** Shrinks triage time from 40 minutes per dropped tool to less than 5 seconds.
- **Elimination of Ambiguity:** Jira tickets include literal orchestrator log timestamps, ending platform vs. customer repo finger-pointing.
- **Cost Elimination:** 100% open-weights architecture running on local consumer laptops with zero external SaaS fees.
- **Safe Extensibility:** Built with graceful degradation—tests complete cleanly even if Ollama is offline or uninstalled.